



UNIVERSIDAD SANTA MARIA

FACULTAD INGENIERIA Y ARQUITECTURA - SISTEMAS

Proyecto de Ingeniería de Software II — Ciberseguridad

Plataforma CTF con Contenedores Dinámicos:

**Diseño, Implementación y Validación de un Orquestador de Docker
con Aislamiento por Usuario y Retos OWASP Top 10**

Integrantes:

Agostini Pedro
Castellanos Daniel
Guillén Arantza
Pérez Samuel
Peña José

Profesor: Alexander Moya

Caracas, Venezuela, septiembre de 2026

Índice

Introducción	4
Capítulo 1. Definición del Proyecto	5
1.1 Descripción general	5
1.2 Problemas a resolver	5
1.3 Objetivos	5
1.4 Alcance	6
Capítulo 2. Marco Teórico	7
2.1 Competencias Capture The Flag como herramienta pedagógica	7
2.2 Contenedores Linux: namespaces y cgroups	7
2.3 Docker Engine API y socket Unix	7
2.4 OWASP Top 10 (2021)	7
2.5 Arquitectura ASGI, WebSockets y emulación de terminal	8
2.6 Principios de diseño seguro	8
2.7 Metodologías iterativas e incrementales	8
Capítulo 3. Análisis de Vulnerabilidad	9
3.1 Modelo de amenazas de la plataforma	9
3.2 Vulnerabilidades pedagógicas: los diez retos OWASP	10
3.3 Hallazgo crítico: bypass trivial de los retos iniciales	10
Capítulo 4. Diseño de la Solución	11
4.1 Arquitectura general	11
4.2 Componentes	11
4.3 Modelo de datos	12
4.4 Flujos principales	13
4.5 Mecanismos de aislamiento y endurecimiento	13
4.6 Decisiones de diseño y alternativas descartadas	14
Capítulo 5. Metodología de Desarrollo	15
5.1 Enfoque adoptado	15
5.2 Iteraciones	15
5.3 Herramientas y entorno	15
Capítulo 6. Implementación y Pruebas	16
6.1 Stack tecnológico	16
6.2 Implementación de los componentes clave	16
6.3 Estrategia y niveles de prueba	17
6.4 Pruebas unitarias automatizadas	17
6.5 Pruebas de aislamiento	18
6.6 Pruebas de estrés y de carga	18

6.7 Pruebas de los retos	20
Capítulo 7. Resultados	21
7.1 Resultados cuantitativos	21
7.2 Resultados cualitativos	21
7.3 Defectos detectados por la campaña de validación	22
Capítulo 8. Conclusiones y Mejoras Futuras	23
8.1 Conclusiones	23
8.2 Limitaciones	23
8.3 Mejoras futuras	24
Referencias	25

Introducción

La formación práctica en ciberseguridad exige entornos donde el estudiante pueda ejecutar ataques reales sobre software deliberadamente vulnerable sin poner en riesgo la infraestructura de la institución ni a los demás participantes. Las competencias de tipo “Capture The Flag” (CTF) son el formato más extendido para este propósito, porque convierten la explotación de una vulnerabilidad en un objetivo concreto y verificable: obtener una cadena secreta, la *flag*, que solo se alcanza al comprometer la falla (Chapman et al., 2014).

Sin embargo, montar un laboratorio de este tipo presenta un problema de ingeniería no trivial. Cada estudiante necesita un entorno propio, aislado de los demás, con límites de recursos que impidan que un error o un abuso (un bucle infinito, una *fork bomb*, etc.) degrade al servidor compartido, y con un ciclo de vida automático que libere recursos cuando el entorno deja de usarse. Los contenedores Linux, y en particular Docker, ofrecen la base técnica para resolverlo, pero su uso seguro en un escenario multiusuario requiere decisiones deliberadas de diseño.

Este trabajo documenta el diseño, la implementación y la validación de Orquestador-daemon, una plataforma CTF con contenedores dinámicos desarrollada como proyecto de la asignatura Ingeniería de Software II. La plataforma permite que cada usuario autenticado despliegue bajo demanda un contenedor aislado con uno de diez retos alineados al OWASP Top 10 (2021), interactúe con él mediante una consola en el navegador, superando el reto al ingresar la “flag” correcta. El sistema destruye la instancia de forma automática por inactividad o por vencimiento de su vida máxima.

El documento se estructura en ocho capítulos. El capítulo 1 define el proyecto, los problemas que resuelve, sus objetivos y su alcance. El capítulo 2 presenta el marco teórico. El capítulo 3 desarrolla el análisis de vulnerabilidad, tanto de las fallas pedagógicas que los retos exponen como de los vectores de ataque contra la propia plataforma. El capítulo 4 describe el diseño de la solución y la propuesta técnica adoptada. El capítulo 5 expone la metodología de desarrollo. El capítulo 6 detalla la implementación, las herramientas empleadas y la campaña de pruebas —unitarias, de humo, de estrés y de carga— realizada sobre el sistema. El capítulo 7 resume los resultados y los defectos detectados durante la validación, y el capítulo 8 presenta las conclusiones, las limitaciones del sistema y las mejoras futuras.

Capítulo 1. Definición del Proyecto

1.1 Descripción general

“Orquestador-daemon” es una aplicación web que actúa como orquestador de contenedores Docker para fines educativos. Su función central es recibir la solicitud de un estudiante autenticado, crear para él una red aislada y un contenedor con la imagen del reto elegido, exponer una consola interactiva sobre ese contenedor a través del navegador, validar la *flag* que el estudiante envía y destruir el entorno cuando corresponde. El código fuente se encuentra en el repositorio público del proyecto (Castellanos et al., 2026).

El proyecto se planteó con tres retos técnicos obligatorios. El primero, de *backend*, exige que el orquestador se comunique directamente con el socket del daemon de Docker, sin usar el SDK oficial `docker-py` ni invocar el binario `docker`. El segundo, de *frontend*, exige integrar Xterm.js para ofrecer una terminal real en el navegador comunicada por WebSockets bidireccionales. El tercero, denominado *modo difícil*, exige aplicar límites estrictos de recursos mediante cgroups y namespaces, y destruir automáticamente las instancias por inactividad.

1.2 Problemas a resolver

El análisis del contexto identificó cinco problemas concretos que la plataforma debe resolver.

1. **Aislamiento entre estudiantes.** Un participante no debe poder alcanzar, observar ni interferir con el entorno de otro, ni salir a Internet desde el contenedor.
2. **Protección del servidor anfitrión.** Un proceso descontrolado dentro de un contenedor (agotamiento de memoria, saturación de CPU, *fork bomb*, etc.) no debe degradar al host ni a los demás contenedores.
3. **Ciclo de vida automático.** Los entornos abandonados deben destruirse sin intervención humana, incluso cuando un proceso en segundo plano simule actividad.
4. **Interacción en tiempo real.** La consola del navegador debe comportarse como una terminal real: soporte de secuencias ANSI, redimensionado, pegado seguro y baja latencia.
5. **Integridad pedagógica.** Cada reto debe resolverse únicamente explotando la vulnerabilidad prevista; no debe existir un atajo como leer la “flag” directamente del sistema de archivos.

1.3 Objetivos

1.3.1 Objetivo general

Diseñar, implementar y validar una plataforma CTF que despliegue contenedores Docker aislados bajo demanda por usuario autenticado, con consola interactiva en el navegador, límites de recursos aplicados por el kernel y destrucción automática de instancias, cubriendo las diez categorías del OWASP Top 10 (2021).

1.3.2 Objetivos específicos

1. Implementar un cliente de la Docker Engine API sobre el socket Unix, incluido el *hijacking* de la conexión para el flujo de la terminal, sin bibliotecas de alto nivel.
2. Construir el orquestador con Django y Django Channels, con un modelo de datos que persista el estado de cada instancia.
3. Integrar Xterm.js con un consumidor WebSocket asíncrono que transporte bytes crudos entre el navegador y el proceso del contenedor.
4. Aplicar aislamiento de red por usuario y límites de memoria, CPU, procesos y descriptors mediante la configuración `HostConfig` del contenedor.

5. Implementar un “watchdog” que destruya instancias por inactividad y por vida máxima según la dificultad del reto.
6. Desarrollar diez retos con separación real de privilegios, uno por categoría del OWASP Top 10, con validación de “flags” y puntaje en el servidor.
7. Verificar empíricamente el aislamiento, los límites y la robustez del sistema mediante una suite de pruebas automatizadas y una campaña de pruebas de humo, de estrés y de carga.

1.4 Alcance

El alcance del producto mínimo viable (MVP) incluye el orquestador completo, la consola interactiva, el aislamiento de red y recursos, el “watchdog” con doble criterio de destrucción, el catálogo de diez retos, la validación de “flags”, un panel de administración con configuración editable en caliente y vista de instancias en vivo. La plataforma se ejecuta en Linux o en Windows con “WSL2”, porque depende del socket Unix de Docker.

Capítulo 2. Marco Teórico

2.1 Competencias Capture The Flag como herramienta pedagógica

Un CTF es un ejercicio en el que los participantes deben encontrar y explotar vulnerabilidades en sistemas preparados para ello, con el fin de obtener “flags” que demuestran el éxito del ataque. Vigna (2003) documentó tempranamente el valor de los ejercicios prácticos en entornos controlados para la enseñanza de seguridad de redes. Por su parte, Chapman et al. (2014) mostraron con PicoCTF que el formato de juego incrementa la motivación y la retención de conceptos.

La condición esencial de un CTF educativo es que la “flag” solo sea alcanzable explotando la falla prevista. Si existe un atajo, el reto deja de medir la competencia que pretende enseñar. Este principio guió la decisión de separar privilegios dentro de cada contenedor, descrita en el capítulo 4.

2.2 Contenedores Linux: namespaces y cgroups

Un contenedor es un conjunto de procesos que el kernel de Linux aísla del resto del sistema mediante dos mecanismos complementarios. Los *namespaces* particionan los recursos visibles: un proceso dentro de un namespace de PID solo ve los procesos de ese namespace, uno dentro de un namespace de red solo ve sus propias interfaces y tablas de rutas, y así con los puntos de montaje, los identificadores de usuario y otros recursos (Kerrisk, 2024a). Los *control groups* (cgroups), por su parte, limitan y contabilizan el uso de recursos (memoria, tiempo de CPU, número de procesos) de un grupo de procesos (Kerrisk, 2024b). Docker combina ambos mecanismos con un sistema de archivos en capas para ofrecer contenedores ligeros y reproducibles (Merkel, 2014).

Las *capabilities* de Linux dividen los privilegios del superusuario en unidades independientes (Kerrisk, 2024c). Docker asigna por defecto un subconjunto reducido, pero la práctica recomendada en entornos multiusuario es retirarlas todas y devolver únicamente las estrictamente necesarias (Docker Inc., 2024b). Esta recomendación no es solo teórica: revisiones sobre seguridad de contenedores documentan que los namespaces y los cgroups no bastan por sí solos cuando el conjunto de capacidades y el perfil de *seccomp* no se configuran de forma restrictiva, porque ambos mecanismos comparten el mismo kernel con el host y una capacidad de más amplia la superficie de ataque disponible (Combe et al., 2016; Sultan et al., 2019).

2.3 Docker Engine API y socket Unix

El daemon de Docker expone una API REST sobre el socket Unix. Toda operación del cliente oficial (crear redes, crear y arrancar contenedores, abrir procesos) corresponde a una petición HTTP/1.1 sobre ese socket (Docker Inc., 2024a).

Para las operaciones de flujo continuo, como adjuntar una terminal a un proceso, la API implementa un mecanismo de *hijacking*: el cliente envía una petición con las cabeceras `Upgrade: tcp` y `Connection: Upgrade`, el daemon responde `101 Switching Protocols`, y a partir de ese momento la misma conexión transporta bytes crudos en ambas direcciones.

2.4 OWASP Top 10 (2021)

El OWASP Top 10 es un documento de concienciación que enumera las diez categorías de riesgo más críticas en aplicaciones web, elaborado a partir de datos de más de 500 000 aplicaciones y de una encuesta a profesionales (OWASP Foundation, 2021). La edición 2021 incluye: A01 Pérdida de control de acceso, A02 Fallas criptográficas, A03 Inyección, A04 Diseño inseguro, A05 Configuración de seguridad incorrecta, A06 Componentes vulnerables y desactualizados, A07 Fallas de identificación y autenticación, A08 Fallas de integridad de software y datos, A09 Fallas de registro y monitoreo, y A10 Falsificación de solicitudes del lado

del servidor (SSRF). Cada categoría se asocia a debilidades del catálogo CWE de MITRE (2023), como CWE-89 para inyección SQL o CWE-502 para deserialización insegura. El catálogo de retos del proyecto cubre las diez categorías.

2.5 Arquitectura ASGI, WebSockets y emulación de terminal

El protocolo WebSocket, definido en la RFC 6455, establece un canal bidireccional persistente sobre una única conexión TCP, iniciado con un *handshake* HTTP (Fette & Melnikov, 2011). En el ecosistema Python, la especificación ASGI generaliza la interfaz WSGI para soportar protocolos de larga duración de forma asíncrona (Django Software Foundation, 2024b). Django Channels implementa ASGI para Django e introduce los *consumers*, clases que manejan conexiones WebSocket; Daphne es el servidor ASGI de referencia (Django Software Foundation, 2024a).

Xterm.js es un emulador de terminal escrito en TypeScript que interpreta secuencias de escape ANSI en el navegador y expone eventos de entrada y redimensionado (Xterm.js Contributors, 2024).

2.6 Principios de diseño seguro

Saltzer y Schroeder (1975) formularon principios que siguen vigentes: el mínimo privilegio (cada componente debe operar con los permisos mínimos necesarios), la mediación completa (toda operación debe verificarse contra la política de acceso), los valores por defecto seguros y la economía de mecanismo. La defensa en profundidad añade que ninguna capa debe ser la única barrera. La guía NIST SP 800-190 traslada estos principios a contenedores: imágenes mínimas, permisos estrictos sobre archivos sensibles, retiro de capacidades, límites de recursos y aislamiento de red (Souppaya et al., 2017).

2.7 Metodologías iterativas e incrementales

Frente a los modelos secuenciales, los enfoques iterativos e incrementales construyen el sistema en ciclos cortos, cada uno con un entregable verificable, lo que permite descubrir problemas temprano y ajustar el diseño con evidencia (Sommerville, 2011; Pressman & Maxim, 2015). El concepto de producto mínimo viable (MVP) propone entregar primero el núcleo funcional imprescindible y ampliarlo con base en lo aprendido (Ries, 2011). Estos ciclos cortos con un entregable verificable comparten, además, el espíritu del Manifiesto Ágil de priorizar el software funcionando por sobre la documentación exhaustiva (Beck et al., 2001), aun cuando el proyecto no adoptó un marco ágil formal. El proyecto adoptó una variante de estos principios con iteraciones diarias documentadas, descrita en el capítulo 5.

Capítulo 3. Análisis de Vulnerabilidad

El análisis de vulnerabilidad se realizó en dos planos. El primero corresponde a las vulnerabilidades pedagógicas: las fallas deliberadas que cada reto expone para que el estudiante las explote. El segundo corresponde a las vulnerabilidades de la plataforma: los vectores por los que un estudiante malintencionado podría atacar al servidor, a otros estudiantes o a la integridad de los retos.

3.1 Modelo de amenazas de la plataforma

El modelado de amenazas identifica de forma sistemática los activos, los actores y los vectores de ataque antes de diseñar las contramedidas (Shostack, 2014). Siguiendo ese enfoque, los activos a proteger en esta plataforma son el servidor anfitrión (memoria, CPU, disco, socket de Docker), el entorno de cada estudiante, la base de datos de la plataforma y la integridad de las “flags” y del puntaje. El actor principal es un estudiante autenticado con acceso a una consola dentro de un contenedor; se asume que puede ejecutar cualquier comando disponible en la imagen y que puede actuar de forma hostil. No se considera un atacante externo sin cuenta, porque toda la API exige autenticación de Django.

Tabla 1

Vectores de ataque contra la plataforma identificados en el análisis

Categoría	Vector	Impacto potencial	Contramedida diseñada
Recursos	Reserva de memoria sin freno	Agotar la RAM del host	Memory 256 MB (cgroup)
Recursos	Bucles de CPU concurrentes	Degradar a otros usuarios	NanoCpus 0,5 núcleos
Recursos	<i>Fork bomb</i>	Agotar la tabla de procesos	PidsLimit 64 + Init: true
Recursos	Apertura masiva de archivos	Agotar descriptores del host	Ulimits nofile 1024/2048
Recursos	Llenar el disco	Inutilizar el host	/tmp en tmpfs de 64 MB; /app de solo lectura por permisos (root:root, 0755)
Recursos	Inundar el log del contenedor	Llenar el disco del host	LogConfig 10 MB × 3
Red	Alcanzar el contenedor de otro usuario	Robo de <i>flags</i> , sabotaje	Red Internal por usuario
Red	Salir a Internet	Exfiltración, descarga de herramientas	Sin gateway (Internal: true)
Red	Alcanzar servicios del host	Atacar la propia plataforma	Sin ruta por defecto; sin docker.sock montado
Escalada	Obtener root vía capabilities	Escape del contenedor	CapDrop: ALL + CapAdd mínimo
Escalada	Injectar módulos vía PYTHONPATH en sudo	Ejecutar código como root	env_reset de sudo
Escalada	Crear <i>user namespace</i>	Ampliar superficie del kernel	Perfil <i>seccomp</i> por defecto de Docker
Integridad	Leer la <i>flag</i> directamente	Anular el reto	Archivo root:600 + sudo restringido
Ciclo de vida	Mantener la instancia viva con salida periódica	Ocupar recursos indefinidamente	Vida máxima absoluta

Categoría	Vector	Impacto potencial	Contramedida diseñada
Ciclo de vida	Dejar procesos huérfanos al cerrar la consola	Agotar PidsLimit propio	Terminación por sesión al desconectar

3.2 Vulnerabilidades pedagógicas: los diez retos OWASP

Cada reto es una pequeña aplicación de consola escrita en Python que contiene una falla real de la categoría OWASP correspondiente. La Tabla 2 resume la vulnerabilidad, su debilidad CWE asociada y la técnica de explotación esperada.

Tabla 2

Catálogo de retos, categoría OWASP, debilidad CWE y técnica de explotación

Reto (slug)	OWASP 2021	Dificultad	Vulnerabilidad y explotación
injection-sqli	A03 Inyección	Básico	Consulta SQL por concatenación (CWE-89). <i>Bypass</i> del login con <code>admin'--</code> .
idor	A01 Control de acceso	Básico	CLI de pedidos sin verificar propietario (CWE-639). Solicitar el pedido 1337 ajeno.
crypto-debil	A02 Fallas criptográficas	Básico	Contraseñas con MD5 sin sal (CWE-328). Ataque de diccionario con la <i>wordlist</i> provista.
misconfig	A05 Configuración incorrecta	Básico	Respaldo <code>config.yaml.bak</code> legible (CWE-538). Leer credenciales expuestas.
diseno-inseguro	A04 Diseño inseguro	Intermedio	Cupón sin límite de usos por compra (CWE-840). Repetir el cupón en una sola invocación.
componente-vulnerable	A06 Componentes vulnerables	Intermedio	<code>libcalc 1.4.2</code> usa <code>eval()</code> sin sandbox (CWE-95). Inyectar una expresión Python.
auth-fallas	A07 Autenticación	Intermedio	PIN de 4 dígitos sin límite de intentos (CWE-307). Fuerza bruta de 10 000 combinaciones.
logging-fallas	A09 Registro y monitoreo	Intermedio	Token de sesión en texto plano en el log (CWE-532). Reutilizar el token.
integridad-datos	A08 Integridad de datos	Difícil	<code>pickle.load()</code> sobre archivo editable (CWE-502). Deserialización con carga maliciosa.
ssrf	A10 SSRF	Difícil	Lista negra sensible a mayúsculas (CWE-918). Evadir el filtro variando el caso del host.

3.3 Hallazgo crítico: bypass trivial de los retos iniciales

Durante la revisión previa a escalar el catálogo, se comprobó que los dos primeros retos podían resolverse sin explotar nada. La consola corría como `root` dentro del contenedor, de modo que `cat /app/flag.txt` entregaba la *flag* de inmediato. Este hallazgo constituye la vulnerabilidad más grave del análisis desde la perspectiva pedagógica, porque anulaba el propósito del sistema. La contramedida adoptada fue la separación real de privilegios: un usuario sin privilegios para la consola, archivos sensibles con permisos 600 de `root`, y un `sudo` restringido al comando exacto de la aplicación vulnerable, de modo que la única vía hacia la *flag* es explotar la falla a través de ese comando.

4.2.4 Catálogo de retos (*challenges.py*)

Es un diccionario en memoria con los diez retos: nombre, categoría OWASP, dificultad, *flag* dinámica, imagen Docker, objetivo, primer paso y resultado esperado. El cliente solo envía el *slug*; la imagen real nunca viaja en la petición, lo que impide que un usuario solicite arrancar una imagen arbitraria del host.

4.2.5 Watchdog (*watchdog.py*, *apps.py*)

La función `sweep_stale_instances()` vive en un único módulo y la usan dos entradas: un hilo integrado que arranca junto al servidor y el comando `manage.py watchdog`. Recorre todas las instancias, calcula para cada una el tiempo inactivo y el tiempo de vida, y destruye las que superan cualquiera de los dos límites. Si Docker no responde, conserva la fila para reintentar en la siguiente pasada, evitando dejar contenedores huérfanos.

4.3 Modelo de datos

Tabla 3

Modelos de datos de la plataforma

Modelo	Campos principales	Propósito
Instance	user (OneToOne), challenge, container_id, network_id, network_name, created_at, last_activity, failed_flag_attempts	Estado de la instancia activa de cada usuario. Se elimina al destruir el contenedor. El contador de intentos fallidos sostiene el límite de cinco intentos de <i>flag</i> (sección 6.2).
SolvedChallenge	user (FK), challenge_slug, solved_at, xp_awarded; único por (usuario, reto)	Registro persistente de retos completados y XP ganado.
PlatformSettings	inactivity_timeout_seconds, max_lifetime_seconds, time_limit_basico/intermedio/difícil_seconds	Configuración editable en caliente desde /admin/ o desde el panel oculto; fila única (<i>singleton</i>).
InstanceDestructionNotice	user (FK), motivo (admin / inactividad / vida_máxima), challenge_slug, created_at	Aviso de un solo uso: informa al estudiante por qué se

Modelo	Campos principales	Propósito
		destruyó su instancia cuando no fue él quien lo pidió.
OnboardingState	user (OneToOne), tutorial_visto, tour_visto, updated_at	Registra si el estudiante ya vio el tutorial y el recorrido guiado, para mostrarlos una sola vez.

La relación uno a uno entre usuario e instancia es una decisión consciente del MVP: simplifica la lógica de estado.

4.4 Flujos principales

Inicio de instancia. El navegador envía `POST /api/instance/start/` con el *slug* del reto. La vista verifica que el usuario no tenga otra instancia, crea la red `ctf-net-{usuario}` con `Internal: true`, crea el contenedor con la imagen del catálogo y el `HostConfig` de la sección 4.5, lo arranca y guarda la fila `Instance`.

Sesión de terminal. Al abrir `ws/terminal/`, el consumidor crea el `exec`, realiza el *hijack* y mantiene dos flujos: del socket al WebSocket (salida del proceso) y del WebSocket al socket (teclas). Los mensajes JSON de tipo `resize` invocan `/exec/{id}/resize`.

Envío de flag. `POST /api/challenges/submit/` compara la cadena enviada con la bandera dinámica generada para ese usuario y ese reto. Si coincide y el usuario no había resuelto ese reto, crea `SolvedChallenge`. Cada intento fallido incrementa `failed_flag_attempts`; al quinto fallo la instancia se destruye.

Destrucción. Ocurre por tres vías: el botón Destruir (`POST /api/instance/stop/`), el *watchdog* (inactividad o vida máxima) y el panel de administración. En todos los casos se detiene y elimina el contenedor, se elimina la red y se borra la fila.

4.5 Mecanismos de aislamiento y endurecimiento

La configuración `HostConfig` común a los diez retos concentra las contramedidas del capítulo 3.

```
"HostConfig": {
  "NetworkMode": network_name,          # red Internal por usuario
  "Memory": 256 * 1024 * 1024,          # 256 MB
  "NanoCpus": 500_000_000,              # 0,5 núcleos
  "PidsLimit": 64,
  "CapDrop": ["ALL"],
  "CapAdd": ["SETUID", "SETGID"],        # solo lo que sudo necesita
  "Tmpfs": {"/tmp": "rw,noexec,nosuid,size=64m"},
  "Ulimits": [{"Name": "nofile", "Soft": 1024, "Hard": 2048}],
  "LogConfig": {"Type": "json-file", "Config": {"max-size": "10m", "max-file": "3"}},
  "Init": True,                          # tini como PID 1: cosecha zombies
}
```

Figura 2. Configuración de creación del contenedor (`HostConfig`).

No se aplica `ReadOnlyRootfs` sobre el contenedor completo, porque el orquestador necesita inyectar la *flag* dinámica del estudiante dentro del contenedor antes de arrancarlo, mediante la Docker Engine API. La protección del sistema de archivos se sostiene entonces por permisos: `/app` y los binarios de cada reto pertenecen a `root:root` con modo 0755, y el archivo con la *flag* tiene modo 600, de manera que el usuario sin privilegios de la consola no puede escribir en `/app` ni leer la *flag* directamente.

A nivel de imagen, todos los retos heredan de `ctf-base` (Alpine con `bash`, `python3` y `sudo`). La consola corre como el usuario sin privilegios `retador`; la *flag* y los datos sensibles pertenecen a `root`; y un archivo en `/etc/sudoers.d/` autoriza sin contraseña exactamente el comando de la aplicación vulnerable. De este modo, la única vía hacia la *flag* es explotar la falla a través de ese comando.

4.6 Decisiones de diseño y alternativas descartadas

Tabla 4

Decisiones arquitectónicas, alternativas descartadas y justificación

Decisión	Alternativa descartada	Justificación
Socket Unix directo + <i>hijacking</i> manual	<code>docker-py</code> ; <code>subprocess</code> con el binario <code>docker</code>	Cumple la restricción del proyecto sin crear un subproceso por usuario ni fugar descriptores.
ASGI (Daphne + Channels)	WSGI síncrono (Gunicorn)	Cada consola es una conexión de larga duración; WSGI bloquearía un <i>worker</i> por sesión.
Red Internal por usuario	Red compartida + reglas <code>iptables</code>	El kernel no crea <i>gateway</i> ; aislamiento sin tocar el firewall del host ni requerir <code>root</code> .
Límites por <code>cgroups</code> en <code>HostConfig</code>	Monitor en Python que mate procesos	El kernel actúa en tiempo real, durante la propia llamada al sistema.
<i>Watchdog</i> con estado en BD	<code>threading.Timer</code> en memoria	Sobrevive reinicios del servidor; no deja contenedores huérfanos.
<code>loop.sock_recv()</code> sin hilos	Hilo bloqueante por consola	Elimina el techo del pool de hilos y permite cancelación inmediata.
<code>InMemoryChannelLayer</code> y <code>SQLite</code>	Redis y PostgreSQL	Suficiente para un proceso y un despliegue de un solo host; el cambio está previsto como mejora futura.

Capítulo 5. Metodología de Desarrollo

5.1 Enfoque adoptado

El proyecto siguió una metodología iterativa e incremental orientada a un MVP, con iteraciones de un día y un entregable verificable al final de cada una (Sommerville, 2011). El punto de partida fue un Documento de Diseño de Software (SDD) que fijó objetivo, alcance, arquitectura, modelo de datos, flujos, decisiones de diseño y exclusiones explícitas. A partir de él se elaboró un plan de siete días para el MVP, que luego se extendió para incorporar el catálogo OWASP, el panel de administración, la auditoría adversarial y, en una fase final, una suite de pruebas automatizadas.

5.2 Iteraciones

Tabla 5

Iteraciones del proyecto, objetivo y resultado verificable

Día	Objetivo	Resultado verificable
1	Cliente de la Docker Engine API sin SDK	Crear, arrancar y destruir contenedores por el socket; <i>hijack</i> del <i>exec</i>
2	Orquestador en Django	Modelo <i>Instance</i> , vistas start/stop, login
3	Aislamiento de red y límites de recursos	Pruebas 1–10: aislamiento y límites contenidos bajo ataque
4	Destrucción automática	<i>Watchdog</i> por inactividad, integrado y como comando
5	Consola WebSocket + Xterm.js	Terminal funcional con <i>resize</i> y <i>bracketed paste</i>
6	Integración y pruebas de fuga	Tres escenarios de abandono y abuso; hueco de salida periódica medido
7	Vida máxima absoluta	Instancia con salida constante destruida
8–9	Retos OWASP	Diez retos con separación de privilegios, 10/10 verificados de punta a punta
10	Guía por reto	Objetivo, primer paso y resultado esperado en el catálogo
11–13	Correcciones de interfaz	<i>SolvedChallenge</i> , validación en servidor, caché de estáticos, modal oculto
14–15	Configuración en caliente y tiempo por dificultad	<i>PlatformSettings</i> editable desde <i>/admin/</i> , reloj en la interfaz
16	Panel de instancias en vivo y auditoría adversarial	4 bugs corregidos, 17 vectores verificados, 4 riesgos aceptados, 0 fallas abiertas
17	Suite de pruebas automatizadas y campaña de validación	84 pruebas unitarias, 10 pruebas de humo, 8 pruebas de estrés con comparativo y 20/20 consolas de carga; 3 defectos nuevos documentados

5.3 Herramientas y entorno

El desarrollo se realizó sobre Ubuntu en WSL2 con Docker Desktop (Engine 28.5.1) y Python 3.11. Las dependencias se fijaron en `requirements.txt`: Django 5, Channels 4, Daphne 4, `requests-unixsocket` 0.3 y `urllib3`. El entorno virtual y la base SQLite se ubican en el sistema de archivos nativo de Linux.

Capítulo 6. Implementación y Pruebas

6.1 Stack tecnológico

Tabla 6

Tecnologías empleadas por capa

Capa	Tecnología	Rol
Frontend	HTML, CSS, JavaScript, Xterm.js con fitAddon	Emulación de terminal, login, selección de retos, envío de <i>flags</i>
Servidor	Django 5, Django Channels 4, Daphne 4	Vistas HTTP, consumidor WebSocket, ASGI, ASGIStaticFilesHandler
Docker	requests-unixsocket, socket de la biblioteca estándar	REST de control y <i>hijacking</i> del exec
Persistencia	SQLite (migraciones de Django)	Instance, SolvedChallenge, PlatformSettings, OnboardingState, InstanceDestructionNotice
Retos	Docker, Alpine, Python 3, sudo	Diez imágenes derivadas de ctf-base
Sistema	Linux (WSL2), cgroups v2, namespaces, <i>seccomp</i>	Aislamiento y límites reales

6.2 Implementación de los componentes clave

6.2.1 Hijacking del exec

La implementación manual del protocolo de *hijack* fue un reto. El cliente envía sobre el socket Unix una petición como la siguiente y lee hasta el fin de las cabeceras; si el estado es 101, el resto del flujo son bytes.

```
POST /exec/{exec_id}/start HTTP/1.1
Host: docker
Content-Type: application/json
Connection: Upgrade
Upgrade: tcp
Content-Length: ...

{"Detach": false, "Tty": true}
```

Figura 3. Petición de hijack.

La primera versión usaba `sync_to_async` con un hilo bloqueado en `recv()` por cada consola. La prueba de carga del día 6 mostró un techo duro cercano al tamaño del *pool* de hilos de Python (`min(32, CPUs + 4)`) y fuga de hilos cuando una conexión moría sin cerrarse. La corrección consistió en poner el socket en modo no bloqueante y leerlo con `loop.sock_recv()`, lo que eliminó el techo y permitió que `task.cancel()` surta efecto de inmediato. La sección 6.6 documenta la medición que confirma esta corrección con veinte consolas simultáneas.

6.2.2 Doble criterio de destrucción

El *watchdog* evalúa dos condiciones independientes para cada instancia: inactividad y vida máxima. La vida máxima se introdujo tras notar que un bucle con salida periódica (`while true; do date; sleep 3; done &)` mantenía `last_activity` fresco indefinidamente.

6.2.3 Terminación por sesión

Al desconectar una consola, `terminate_console()` agrupa los procesos del contenedor por identificador de sesión y los termina, para no dejar procesos huérfanos consumiendo el cupo de `PidsLimit` del propio estudiante.

6.3 Estrategia y niveles de prueba

La verificación del sistema se organizó en cuatro niveles, de menor a mayor contacto con el entorno real. Los dos primeros responden si el sistema está bien construido; los dos últimos, si resiste el uso bajo condiciones adversas. El criterio transversal, aplicado desde la auditoría del día 16, fue no comprobar únicamente que el sistema funciona, sino intentar romperlo de forma deliberada.

Tabla 7

Niveles de verificación aplicados al sistema

Nivel	Qué verifica	Entorno	Casos
Unitario	Lógica del orquestador aislada: catálogo, banderas, modelos, API y <i>watchdog</i>	Docker sustituido por dobles de prueba	84
Humo	Ciclo de vida completo de una instancia contra el motor real	Docker Engine real	10
Estrés	Resistencia del aislamiento ante ataques ejecutados desde el contenedor	Docker real, dos configuraciones por prueba	8
Carga	Comportamiento del sistema con veinte estudiantes simultáneos	Servidor Daphne y veinte contenedores reales	7 fases

En el nivel de estrés, cada ataque se ejecuta dos veces: primero sobre un contenedor sin los límites de la plataforma y después sobre uno con la configuración real descrita en la sección 4.5. De este modo la contención que ofrece el kernel queda medida por contraste y no solamente afirmada. La campaña completa se ejecutó sobre un equipo con 12 núcleos y 7,6 GB de RAM, bajo WSL2 con Docker Engine 28.5.1, contra la revisión del repositorio identificada como `33cf672`.

6.4 Pruebas unitarias automatizadas

Se construyó una suite de ochenta y cuatro casos en el paquete `ctf/tests/`, organizada en cinco módulos. Ninguno requiere Docker: las llamadas al motor de contenedores se sustituyen por dobles de prueba (*mocks*), de modo que la suite se ejecuta en cualquier equipo, sin depender de un daemon de Docker disponible, y puede incorporarse a un flujo de integración continua. La ejecución completa (`python manage.py test ctf.tests`) toma 8,35 segundos y no registra fallos.

Tabla 8

Módulos de la suite unitaria y riesgo que cubre cada uno

Módulo	Riesgo que cubre	Casos
<code>test_challenges</code>	Que un estudiante acredite un reto sin explotar la vulnerabilidad, o que el catálogo público filtre una bandera	20
<code>test_views</code>	Que la API atienda a un usuario no autenticado, o que un fallo de Docker a mitad del despliegue deje recursos sin liberar	23
<code>test_seguridad</code>	Que una modificación posterior elimine un límite de recursos sin que nadie lo advierta	16
<code>test_models</code>	Que un estudiante acumule más de una instancia o duplique su progreso	13

Módulo	Riesgo que cubre	Casos
test_watchdog	Que las instancias abandonadas no se liberen, o que un fallo de Docker deje contenedores huérfanos	12

El módulo `test_seguridad` cumple una función particular: intercepta la petición que el orquestador envía al socket de Docker y comprueba que contenga los límites de memoria, procesador, procesos, descriptores, capacidades y montajes descritos en la sección 4.5. Opera como salvaguarda ante regresiones, porque un cambio que suprima alguno de esos parámetros —por ejemplo, `PidsLimit` o la red `Internal`— hace fallar la suite antes de llegar a producción. El módulo `test_views` verifica además la regla de negocio más delicada de la API: si Docker falla a mitad de un despliegue, la vista debe eliminar cualquier red o contenedor que ya se hubiera creado, para no dejar recursos huérfanos sin una fila que los reclame; y el módulo `test_watchdog` verifica la contraparte de esa misma regla, que si el fallo ocurre durante la destrucción, la fila debe conservarse para reintentar en el siguiente barrido.

6.5 Pruebas de aislamiento

Antes de las diez pruebas de aislamiento, se verificó el ciclo de vida completo de una instancia contra el motor real de Docker, sin dobles de prueba: creación de la red interna, creación y arranque del contenedor, inyección de la bandera dinámica del estudiante, lectura de los límites desde la interfaz de `cgroups` del propio contenedor, apertura del `exec` de la consola, aparición del contenedor en el inventario que usa el panel de administración, y destrucción sin residuos en el host. Las diez comprobaciones resultaron correctas.

Dos de esas pruebas de humo confirman empíricamente la integridad pedagógica descrita en la sección 3.3: el archivo que contiene la bandera del reto *idor* no es legible con `cat` por el usuario sin privilegios, que recibe “permiso denegado”; en cambio, la bandera sí se obtiene al ejecutar la aplicación vulnerable a través del `sudo` restringido descrito en la sección 4.5. La Tabla 9 recoge las diez pruebas de aislamiento y de límites de recursos ejecutadas contra un contenedor de estudiante.

Tabla 9
Pruebas de aislamiento y de límites de recursos

N.º	Prueba	Resultado
1–2	Dos usuarios con instancias simultáneas, red y contenedor propios	Correcto; redes separadas
3	Alcanzar el contenedor del otro usuario (<i>ping</i> , <i>wget</i> , TCP directo, ruta manual)	Bloqueado: <code>Network unreachable</code>
4	Escanear vecinos de la propia red	Solo el propio contenedor
5	Salida a Internet (<i>wget http://1.1.1.1</i>)	Bloqueado: sin ruta por defecto
6	Alcanzar servicios del host y la plataforma	Sin servicios alcanzables; <code>docker.sock</code> no montado
7	Reservar memoria sin freno	Contenido: contenedor vivo
8	Cuatro bucles infinitos de CPU	Contenido: uso topado en 49–55 %
9	<i>Fork bomb</i> : <code>() { : :& } ; :</code>	Contenido: <code>can't fork</code> , host intacto
10	Estado del host y del testigo	Sin degradación: 8–19 ms en todo momento

6.6 Pruebas de estrés y de carga

Las pruebas de estrés repiten ocho de los ataques anteriores en dos configuraciones —sin los límites de la plataforma y con ellos— para medir la contención por contraste. Se ejecutaron sobre la imagen base del catálogo (`ctf-base:latest`).

Tabla 10

Comparativo de las pruebas de estrés, sin límites y con la configuración de la plataforma

Ataque	Sin los límites	Con la plataforma	Qué lo impide
Bomba de procesos (200 solicitados)	200 procesos creados	62 procesos; bloqueado por el kernel	<code>cgroup pids</code> (<code>PidsLimit 64</code>)
Agotar memoria (pedir 400 MB)	400 MB asignados	Detenido con código 137 (OOM)	<code>cgroup memory</code> + tope de <code>swap</code>
Saturar CPU (bucle infinito)	100,25 % de uso	50,02 % de uso	Planificador CFS (<code>NanoCpus</code>)
Salir a Internet	Conectado a 1.1.1.1	<code>Network unreachable</code>	Red Internal
Alcanzar a otro estudiante	Servicio ajeno visible	<code>Network unreachable</code>	Una red por usuario
Privilegios de <code>root</code>	14 capacidades activas; <code>chown</code> permitido	2 capacidades activas; <code>chown</code> denegado	<code>CapDrop: ALL</code>
Descriptores de archivo	1 048 576 disponibles	1 024 disponibles	<code>Ulimits nofile</code>
Perfil completo aplicado	—	6 de 6 límites verificados juntos	<code>cgroups</code> + <code>namespaces</code>

En los ocho casos, quien detiene el ataque es el kernel de Linux, no código de la aplicación: el planificador y los *cgroups* deciden durante la propia llamada al sistema, antes de conceder el recurso, lo que confirma en la práctica el principio de mínimo privilegio descrito en la sección 2.6. La prueba de memoria reveló además que el límite declarado de 256 MB es, sin un tope explícito de *swap*, efectivamente el doble en la práctica; este hallazgo se documenta en la sección 7.3.

La prueba de carga somete al sistema a veinte estudiantes simultáneos que se autentican, despliegan su instancia, abren la consola, ejecutan una orden y liberan los recursos, contra un servidor Daphne real y veinte contenedores verdaderos. La Tabla 11 registra la medición original del día 6, que motivó la corrección descrita en la sección 6.2.1.

Tabla 11

Prueba de carga con veinte consolas simultáneas, antes y después de la corrección (medición del día 6)

Medición	Antes	Después
Consolas que ejecutaron el comando	0 / 20	20 / 20
Hilos del servidor	16 (tope del <i>pool</i>)	14
Estado de los hilos	<code>unix_stream_read_generic</code> (bloqueados)	<code>futex_wait_queue</code> (ociosos)
Sockets a Docker sin consolas abiertas	25 (filtrados)	3
POST <code>/api/instance/start/</code>	Timeout a los 30 s	14 ms
Fugas de procesos en 25 reconexiones	1 por reconexión	0

La campaña de esta sesión repitió la prueba de carga en un equipo distinto, con veinte estudiantes simulados desplegando el reto `injection-sqli`. Las veinte instancias se crearon correctamente en 7,79 segundos (latencia media de 5,71 s por instancia) y las veinte consolas WebSocket ejecutaron la orden enviada, con una latencia media de 1,63 s. La medición de hilos del servidor, tomada tanto durante el arranque de las consolas como con las veinte ya abiertas y en reposo, no registró ningún hilo bloqueado en entrada o salida de socket: los diecisiete hilos presentes permanecieron en estado `futex_wait_queue`, es decir, ociosos. El dato resulta concluyente porque ese equipo dispone de doce núcleos, de modo que el *pool* de hilos de Python admite un máximo de dieciséis; si cada consola retuviera un hilo propio, veinte sesiones simultáneas serían aritméticamente imposibles. Al finalizar, las veinte instancias se liberaron en 12,40 segundos y no quedó ninguna fila huérfana en la base de datos.

6.7 Pruebas de los retos

Cada reto se verificó en cuatro pasos con `docker run` aislado: el *bypass* directo (`cat flag.txt`) falla; ejecutar la aplicación sin `sudo` falla porque no puede abrir el archivo protegido; el uso normal con `sudo` no entrega la *flag*; y solo la explotación de la vulnerabilidad la revela. Después se probó cada reto de punta a punta contra la plataforma real: inicio de sesión, selección del *slug*, `start_instance`, consola WebSocket, comandos del estudiante, obtención de la *flag* y destrucción.

Capítulo 7. Resultados

La Tabla 12 contrasta cada objetivo específico con la evidencia que respalda su cumplimiento.

Tabla 12

Cumplimiento de los objetivos específicos

Objetivo específico	Estado	Evidencia
Cliente de la Docker Engine API sin SDK, con <i>hijacking</i>	Cumplido	<code>docker_client.py</code> : REST sobre socket Unix y <code>HijackedExecSocket</code> sin hilos
Orquestador Django con estado persistente	Cumplido	Cinco modelos (<code>Instance</code> , <code>SolvedChallenge</code> , <code>PlatformSettings</code> , <code>OnboardingState</code> , <code>InstanceDestructionNotice</code>); nueve migraciones
Consola Xterm.js con WebSocket asíncrono	Cumplido	<code>TerminalConsumer</code> ; 20/20 consolas simultáneas verificadas en dos campañas; <i>bracketed paste</i> ; reconexión sin fugas
Aislamiento de red y límites de recursos	Cumplido	Pruebas 1–10 del día 3; auditoría del día 16; comparativo de estrés de la sección 6.6
<i>Watchdog</i> por inactividad y vida máxima	Cumplido	Destrucción verificada a los 61 s y 34 s en los escenarios de fuga; 12 pruebas unitarias del módulo <code>test_watchdog</code>
Diez retos OWASP con separación de privilegios y puntaje	Cumplido	10/10 verificados de punta a punta; validación de bandera dinámica en servidor
Verificación empírica de aislamiento, límites y robustez	Cumplido	84 pruebas unitarias, 10 de humo, 8 de estrés y 20/20 de carga; 3 defectos nuevos detectados y documentados (sección 7.3)

7.1 Resultados cuantitativos

- Creación de instancia: ~0,87 s por usuario (0,77–1,01 s) en la medición original; 5,71 s de media en la campaña de carga de veinte usuarios simultáneos de esta sesión.
- Consolas simultáneas: 20 de 20 funcionales en dos campañas independientes, sin techo por hilos; cero hilos bloqueados en entrada o salida de socket con las veinte consolas abiertas.
- Latencia del host bajo ataque simultáneo de tres usuarios: 8–19 ms, sin degradación.
- Límites verificados: 268 435 456 B de memoria, 49–55 % de CPU, 64 procesos, ~1024 descriptores, 64 MB en `/tmp`.
- Fugas de procesos en 25 reconexiones: 0. Huérfanos tras la corrida completa de retos: 0.
- Cobertura OWASP: 10 de 10 categorías, con dificultad y guía por reto.
- Suite automatizada: 84 casos unitarios ejecutados en 8,35 s, sin fallos ni dependencia de Docker.
- Verificación contra Docker real: 10 de 10 pruebas de humo correctas, sin residuos en el host tras la destrucción.
- Contención de estrés medida por contraste: 200 procesos sin límite frente a 62 con él; 100,25 % de CPU frente a 50,02 %; salida a Internet y acceso al entorno de otro estudiante bloqueados en ambos casos con la configuración de la plataforma.

7.2 Resultados cualitativos

El sistema cumple los tres retos planteados: *backend* sin SDK, *frontend* con terminal real y *modo difícil* con contención por kernel y destrucción automática. Más allá de lo exigido, incorporó un catálogo completo alineado a OWASP, puntaje persistente, configuración en caliente y un panel de instancias en vivo. El proceso dejó como subproducto una documentación extensa —dieciséis reportes de iteración más la campaña de pruebas automatizadas descrita en el capítulo 6— que registra no solo lo que funciona, sino también lo que falló, cómo se midió y cómo se corrigió.

7.3 Defectos detectados por la campaña de validación

La campaña de pruebas de esta sesión no se limitó a confirmar el comportamiento esperado: detectó tres defectos que la configuración declarada de la plataforma ocultaba. Ninguno de los tres compromete el aislamiento entre estudiantes verificado en la sección 6.5; los tres afectan la capacidad real del sistema o el aprovechamiento de sus recursos.

Tabla 13

Defectos detectados, evidencia medida y corrección propuesta

Defecto	Evidencia medida	Corrección propuesta
El techo de instancias simultáneas lo impone la red de Docker, no la memoria del host	Al elevar la prueba de carga a treinta estudiantes, seis despliegues fallaron con error 500. Docker agota sus rangos de direcciones predefinidos alrededor de la red número veinticuatro: <code>all predefined address pools have been fully subnetted</code>	Declarar un rango de direcciones propio en <code>daemon.json</code> (por ejemplo, base <code>10.200.0.0/16</code> con máscara <code>/24</code> , que habilita 256 redes)
El límite de memoria efectivo por contenedor duplica al declarado	Con el límite de 256 MB vigente, un contenedor asignó 400 MB sin ser detenido; al declarar también el tope de <i>swap</i> , el kernel detuvo la asignación con código 137	Añadir <code>"MemorySwap"</code> : <code>mem_limit_bytes</code> a la configuración de creación del contenedor
La destrucción de cada instancia desperdicia cinco segundos	El proceso principal del contenedor no atiende <code>SIGTERM</code> , de modo que Docker agota el plazo de gracia completo antes de forzar el cierre: 5,30 s frente a 0,37 s sin esa espera	Reducir el plazo de gracia en <code>stop_container()</code>

El primer defecto corrige, además, la estimación de capacidad ofrecida en el capítulo 8 de versiones anteriores de este documento: el límite práctico de estudiantes concurrentes con la configuración predeterminada de Docker no son los contenedores que permitiría la memoria del host, sino las redes que el *daemon* puede asignar, del orden de veinticuatro a treinta y una con esa configuración.

Capítulo 8. Conclusiones y Mejoras Futuras

8.1 Conclusiones

Primero, es posible construir un orquestador de contenedores funcional y seguro hablando directamente con la Docker Engine API. La restricción de no usar SDK, lejos de ser un obstáculo artificial, obligó a comprender el protocolo de *hijacking* y condujo a una implementación más ligera que la alternativa por subprocesos.

Segundo, el aislamiento efectivo en un entorno multiusuario no proviene de una sola medida sino de la superposición de varias: red interna sin *gateway*, *cgroups* para memoria, CPU y procesos, retiro de capacidades, permisos estrictos sobre los datos sensibles, límites de descriptores y un *init* real. La auditoría adversarial y la campaña de pruebas de estrés mostraron que cada capa cubre un vector que las demás no cubren, en línea con el principio de defensa en profundidad (Saltzer & Schroeder, 1975; Souppaya et al., 2017).

Tercero, la verificación empírica descubrió defectos que la configuración declarada ocultaba: el techo del *pool* de hilos, la instancia inmortal por salida periódica, el *bypass* de los retos con *cat*, los zombis de la *fork bomb*, los huérfanos al cerrar la consola y, en la campaña más reciente, el límite real de memoria duplicado por el *swap* y el techo de concurrencia impuesto por las redes de Docker. Ninguno de ellos habría aparecido sin atacar el sistema de forma deliberada. Esta conclusión respalda la práctica de cerrar cada iteración con pruebas que intenten romper lo construido, y de automatizar esas pruebas para que se repitan sin esfuerzo adicional.

Cuarto, la integridad pedagógica de un CTF depende de la separación de privilegios dentro del contenedor. El patrón adoptado —usuario sin privilegios, dato sensible de *root* con modo 600 y *sudo* restringido al comando exacto— es reutilizable para cualquier reto futuro, y quedó verificado tanto manualmente como por la suite automatizada del módulo *test_challenges*.

Quinto, la metodología iterativa con reportes diarios produjo una trazabilidad completa entre requisitos, decisiones, pruebas y correcciones, lo que facilitó tanto el desarrollo como la elaboración de este documento.

8.2 Limitaciones

El sistema se diseñó y se validó como un despliegue de un solo anfitrión, con un único proceso de servidor, un único demonio de Docker y un archivo SQLite. De ese supuesto se derivan las limitaciones siguientes.

Primero, la concurrencia real está acotada por los rangos de direcciones que Docker asigna a sus redes antes que por la memoria del equipo, según se documenta en la sección 7.3: entre veinticuatro y treinta y un estudiantes simultáneos con la configuración predeterminada del daemon.

Segundo, no existe un límite global de instancias a nivel de plataforma, sino únicamente la restricción de una instancia activa por usuario. El total de contenedores desplegados a la vez es, en la práctica, lo que el anfitrión resista.

Tercero, SQLite serializa las escrituras y el consumidor de la terminal actualiza *last_activity* durante toda la sesión; con concurrencia suficiente, esa contención de escritura puede convertirse en el cuello de botella del sistema antes que el cómputo o la red.

Cuarto, *InMemoryChannelLayer* y el hilo integrado del *watchdog* impiden ejecutar más de un proceso de servidor sin trabajo adicional de configuración, lo que limita el escalado horizontal de la capa web.

Quinto, las mediciones de aislamiento de red se realizaron sobre WSL2 con Docker Desktop. La prueba de acceso a servicios del host (prueba 6, sección 6.5) no alcanzó ningún servicio por una particularidad de ese entorno, y debe repetirse en un servidor Linux de producción antes de tomar ese resultado como definitivo.

8.3 Mejoras futuras

1. **Ampliar el espacio de direcciones de Docker y fijar un límite global de instancias.** La configuración predeterminada del daemon agota sus rangos de red alrededor de la instancia número veinticuatro (sección 7.3), lo que acota la concurrencia por debajo de lo que permitiría la memoria disponible. Declarar un rango de direcciones propio en `daemon.json` eleva ese techo a 256 redes; hecho eso, conviene fijar además un límite global de instancias simultáneas, del que hoy se carece.
2. **PostgreSQL y Redis.** SQLite serializa cada escritura y cada tecla actualiza `last_activity`; `InMemoryChannelLayer` no escala a varios procesos. Ambos cambios están anticipados en el SDD.
3. **Perfil *seccomp* personalizado.** Hoy se usa el perfil por defecto de Docker. Un perfil más estricto reduciría aún más la superficie del kernel expuesta.
4. **Manejo de errores cuando Docker no responde.** El error 500 actual es crudo; conviene detectar el estado del daemon y comunicarlo con claridad al usuario.
5. **Ranking y métricas de aprendizaje.** El modelo `SolvedChallenge` permite construir tablas de posiciones, tiempos de resolución y estadísticas por categoría OWASP para el docente.
6. **Instancias múltiples por usuario.** Sustituir la relación uno a uno por una relación uno a muchos con un tope configurable.
7. **Reverificación del acceso al host en despliegue real.** La prueba 6 del día 3 no alcanzó servicios del host por una particularidad de Docker Desktop; debe repetirse en un servidor Linux de producción.
8. **Declarar el tope de intercambio de memoria (*swap*).** El límite vigente permite que cada contenedor duplique su cuota de memoria usando *swap* del host, según se documenta en la sección 7.3.
9. **Acelerar la destrucción de instancias.** Reducir el plazo de gracia de la detención de contenedores recorta el tiempo de liberación de 5,30 s a 0,37 s por instancia (sección 7.3).

Referencias

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. <https://agilemanifesto.org/>
- Castellanos, D., Pérez, S., Guillén, A., Agostini, P., & Peña, J. (2026). *Orquestador-daemon: Plataforma CTF con contenedores dinámicos* [Código fuente y documentación]. GitHub. <https://github.com/Daniel132005/Orquestador-daemon>
- Chapman, P., Burket, J., & Brumley, D. (2014). PicoCTF: A game-based computer security competition for high school students. En *Proceedings of the 2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*. USENIX Association. <https://www.usenix.org/conference/3gse14/summit-program/presentation/chapman>
- Combe, T., Martin, A., & Di Pietro, R. (2016). To Docker or not to Docker: A security perspective. *IEEE Cloud Computing*, 3(5), 54–62. <https://doi.org/10.1109/MCC.2016.100>
- Django Software Foundation. (2024a). *Django Channels documentation* (Versión 4). <https://channels.readthedocs.io/>
- Django Software Foundation. (2024b). *ASGI (Asynchronous Server Gateway Interface) specification* (Versión 3.0). <https://asgi.readthedocs.io/>
- Django Software Foundation. (2024c). *Django documentation* (Versión 5.0). <https://docs.djangoproject.com/en/5.0/>
- Docker Inc. (2024a). *Docker Engine API reference* (Versión 1.45). <https://docs.docker.com/engine/api/>
- Docker Inc. (2024b). *Docker Engine security*. <https://docs.docker.com/engine/security/>
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6455>
- Kerrisk, M. (2024a). *namespaces(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- Kerrisk, M. (2024b). *cgroups(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- Kerrisk, M. (2024c). *capabilities(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), Artículo 2.
- MITRE Corporation. (2023). *Common Weakness Enumeration (CWE)* (Versión 4.13). <https://cwe.mitre.org/>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8.^a ed.). McGraw-Hill Education.
- Ries, E. (2011). *The lean startup: How today's entrepreneurs use continuous innovation to create radically successful businesses*. Crown Business.

- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Souppaya, M., Morello, J., & Scarfone, K. (2017). *Application container security guide* (NIST Special Publication 800-190). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-190>
- Sultan, S., Ahmad, I., & Dimitriou, T. (2019). Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7, 52976–52996. <https://doi.org/10.1109/ACCESS.2019.2911732>
- Vigna, G. (2003). Teaching hands-on network security: Testbeds and live exercises. *Journal of Information Warfare*, 2(3), 8–24.
- Xterm.js Contributors. (2024). *xterm.js: A terminal for the web*. <https://xtermjs.org/>